Lab8_NGram_Model_

IMPORT REQUIRED LIBRARIES

```
import re                       #removing punctuation, numbers, etc
import math                     # Used for probability and perplexity calculations
from collections import Counter, defaultdict  # Counter for counting N-grams, defaultdict for handling missing keys
import nltk                     # Natural Language Toolkit for NLP tasks
from nltk.tokenize import sent_tokenize, word_tokenize  # For sentence and word tokenization
from nltk.corpus import stopwords  # Used for optional stopword removal
import pandas as pd             # Used to create tables for counts and probabilities

# Download required NLTK resources
nltk.download('punkt')          # Required for sentence and word tokenization
nltk.download('stopwords')      # Required for stopword removal
nltk.download('punkt_tab')      # Additional tokenizer data (fixes missing resource error)

from collections import Counter  # Imported again for unigram, bigram, trigram counting
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Load dataset

```
text = """
The dataset used in this experiment is a general English text corpus consisting of more than 1500 words.
It contains paragraphs related to language, communication, natural language processing, language models, and artificial inte
The text is written in clear, grammatically correct English and includes multiple sentences with common vocabulary and sente
This makes it suitable for building Unigram, Bigram, and Trigram language models, as it provides sufficient word combination
The dataset does not contain special symbols or code-mixed language, which simplifies preprocessing and probability calculat
"""

print("Total number of words in dataset:", len(text.split()))

print("\nSample text from dataset:\n")
print(text)
```

```
Total number of words in dataset: 89

Sample text from dataset:


The dataset used in this experiment is a general English text corpus consisting of more than 1500 words.
It contains paragraphs related to language, communication, natural language processing, language models, and artificial inte
The text is written in clear, grammatically correct English and includes multiple sentences with common vocabulary and sente
This makes it suitable for building Unigram, Bigram, and Trigram language models, as it provides sufficient word combinatior
The dataset does not contain special symbols or code-mixed language, which simplifies preprocessing and probability calculat
```

Preprocess Text

```
# Function to convert text to lowercase
def to_lowercase(text):
    return text.lower()


# Function to remove punctuation and numbers
def remove_punctuation_numbers(text):
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    return text


# Function to tokenize words
def tokenize_text(text):
    sentences = sent_tokenize(text)
    tokenized_sentences = []

    for sentence in sentences:
        words = word_tokenize(sentence)
```

```
        tokenized_sentences.append(words)

    return tokenized_sentences


# Optional function to remove stopwords
def remove_stopwords(tokenized_sentences):
    stop_words = set(stopwords.words('english'))
    filtered_sentences = []

    for sentence in tokenized_sentences:
        filtered = [word for word in sentence if word not in stop_words]
        filtered_sentences.append(filtered)

    return filtered_sentences


# Function to add start and end tokens
def add_start_end_tokens(tokenized_sentences):
    final_sentences = []

    for sentence in tokenized_sentences:
        final_sentences.append(['<s>'] + sentence + ['</s>'])

    return final_sentences

# Apply preprocessing steps to the training text
processed_text_lower = to_lowercase(g_train_text)
processed_text_cleaned = remove_punctuation_numbers(processed_text_lower)
tokens_raw = tokenize_text(processed_text_cleaned)
tokens_filtered = remove_stopwords(tokens_raw)      # Optional step
tokens = add_start_end_tokens(tokens_filtered)

# Print preprocessed text sentence by sentence
print("Preprocessed Text:\n")

for i, sentence in enumerate(tokens):
    print(f"Sentence {i+1}:")
    print(" ".join(sentence))
    print()
```

```
Preprocessed Text:

Sentence 1:
<s> natural language processing nlp subfield linguistics computer science artificial intelligence data science goal enable c
```

Build N-Gram Models

```
print("1️⃣ Unigram Model")
# Flatten tokens
all_words = [word for sentence in tokens for word in sentence]

# Count unigrams
unigram_counts = Counter(all_words)

# Convert to table
unigram_table = pd.DataFrame(
    unigram_counts.items(),
    columns=["Word", "Count"]
).sort_values(by="Count", ascending=False)

unigram_table.head()



total_words = sum(unigram_counts.values())

unigram_table["Probability"] = unigram_table["Count"] / total_words

unigram_table.head()
```

1️⃣ Unigram Model

	Word	Count	Probability
4	nlp	112	0.053794
2	language	48	0.023055
55	text	48	0.023055
51	models	40	0.019212
58	like	32	0.015370

```
print("2️⃣ Bigram Model")                # Display model name

bigram_counts = Counter()               # Initialize Counter to store bigram frequencies

# Loop through each preprocessed sentence
for sentence in tokens:
    # Loop through words in the sentence to form bigrams
    for i in range(len(sentence) - 1):
        bigram = (sentence[i], sentence[i + 1])  # Create a bigram (pair of consecutive words)
        bigram_counts[bigram] += 1               # Increment bigram count

# Convert bigram counts into a DataFrame (table format)
bigram_table = pd.DataFrame(
    [(w1, w2, count) for (w1, w2), count in bigram_counts.items()],
    columns=["Word_1", "Word_2", "Count"]
)

bigram_table.head()                     # Display first few rows of bigram count table

# Calculate conditional probability P(Word_2 | Word_1)
bigram_table["Conditional_Probability"] = bigram_table.apply(
    lambda row: row["Count"] / unigram_counts[row["Word_1"]],  # Bigram count / Unigram count
    axis=1
)

bigram_table.head()                     # Display table with conditional probabilities
```

2️⃣ Bigram Model

	Word_1	Word_2	Count	Conditional_Probability
0	<s>	natural	1	1.000000
1	natural	language	16	1.000000
2	language	processing	16	0.333333
3	processing	nlp	16	0.666667
4	nlp	subfield	8	0.071429

```
print("3️⃣ Trigram Model")                # Display model name

trigram_counts = Counter()               # Initialize Counter to store trigram frequencies

# Loop through each preprocessed sentence
for sentence in tokens:
    # Loop through words to create trigrams (3 consecutive words)
    for i in range(len(sentence) - 2):
        trigram = (sentence[i], sentence[i + 1], sentence[i + 2])  # Create a trigram
        trigram_counts[trigram] += 1                               # Increment trigram count

# Convert trigram counts into a DataFrame (table format)
trigram_table = pd.DataFrame(
    [(w1, w2, w3, count) for (w1, w2, w3), count in trigram_counts.items()],
    columns=["Word_1", "Word_2", "Word_3", "Count"]
)

trigram_table.head()                     # Display first few rows of trigram count table

# Calculate conditional probability P(Word_3 | Word_1, Word_2)
trigram_table["Conditional_Probability"] = trigram_table.apply(
    lambda row: row["Count"] / bigram_counts[(row["Word_1"], row["Word_2"])],
    axis=1
)

trigram_table.head()                     # Display table with conditional probabilities
```

3 Trigram Model

	Word_1	Word_2	Word_3	Count	Conditional_Probability
0	<s>	natural	language	1	1.0
1	natural	language	processing	16	1.0
2	language	processing	nlp	16	1.0
3	processing	nlp	subfield	8	0.5
4	nlp	subfield	linguistics	8	1.0

Apply Smoothing

```
# Vocabulary size
V = len(unigram_counts)

# Smoothed Bigram Probability
def bigram_probability_laplace(w1, w2):
    return (bigram_counts[(w1, w2)] + 1) / (unigram_counts[w1] + V)


# Smoothed Trigram Probability
def trigram_probability_laplace(w1, w2, w3):
    return (trigram_counts[(w1, w2, w3)] + 1) / (bigram_counts[(w1, w2)] + V)
    print("Smoothed Bigram Probability:")
print("P(language | natural) =",
      bigram_probability_laplace("natural", "language"))

print("\nSmoothed Trigram Probability:")
print("P(processing | language natural) =",
      trigram_probability_laplace("language", "natural", "processing"))
```

```
P(language | natural) = 0.0821256038647343

Smoothed Trigram Probability:
P(processing | language natural) = 0.005050505050505051
```

Sentence Probability Calculation

```
# List of test sentences for probability calculation
sentences = [
    """The dataset used in this experiment is a general English text corpus consisting of more than 1500 words.
It contains paragraphs related to language, communication, natural language processing, language models, and artificial int
The text is written in clear, grammatically correct English and includes multiple sentences with common vocabulary and sent
This makes it suitable for building Unigram, Bigram, and Trigram language models, as it provides sufficient word combinatic
The dataset does not contain special symbols or code-mixed language, which simplifies preprocessing and probability calcula
]

# Function to calculate sentence probability using the Unigram model
def unigram_sentence_probability(sentence):
    words = word_tokenize(sentence.lower())   # Tokenize sentence into lowercase words
    probability = 1                           # Initialize probability

    for word in words:
        # Multiply unigram probabilities of individual words
        probability *= unigram_counts[word] / total_words

    return probability                        # Return final unigram probability


# Function to calculate sentence probability using the Bigram model with Laplace smoothing
def bigram_sentence_probability(sentence):
    # Add start and end tokens to the sentence
    words = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    probability = 1                           # Initialize probability

    for i in range(len(words) - 1):
        # Multiply smoothed bigram probabilities
        probability *= bigram_probability_laplace(words[i], words[i+1])

    return probability                        # Return final bigram probability


# Function to calculate sentence probability using the Trigram model with Laplace smoothing
def trigram_sentence_probability(sentence):
    # Add two start tokens and one end token
```

```
    words = ['<s>', '<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    probability = 1                          # Initialize probability

    for i in range(2, len(words)):
        # Multiply smoothed trigram probabilities
        probability *= trigram_probability_laplace(
            words[i-2], words[i-1], words[i]
        )

    return probability                       # Return final trigram probability


# Calculate and display probabilities for each sentence
for s in sentences:
    print("Sentence:", s)
    print("Unigram Probability:", unigram_sentence_probability(s))
    print("Bigram Probability:", bigram_sentence_probability(s))
    print("Trigram Probability:", trigram_sentence_probability(s))
    print()
```

```
Sentence: The dataset used in this experiment is a general English text corpus consisting of more than 1500 words.
It contains paragraphs related to language, communication, natural language processing, language models, and artificial inte
The text is written in clear, grammatically correct English and includes multiple sentences with common vocabulary and sente
This makes it suitable for building Unigram, Bigram, and Trigram language models, as it provides sufficient word combinatior
The dataset does not contain special symbols or code-mixed language, which simplifies preprocessing and probability calculat
Unigram Probability: 0.0
Bigram Probability: 2.0789025263605845e-231
Trigram Probability: 1.4477014552448537e-234
```

Perplexity Calculation

```
test_sentences = sent_tokenize(g_test_text) # Define test_sentences from the g_test_text variable

def unigram_perplexity(sentence):
    words = word_tokenize(sentence.lower())
    N = len(words)
    log_prob = 0

    for word in words:
        prob = unigram_counts[word] / total_words
        # Handle log(0) if word not in unigram_counts.
        # A proper unigram perplexity would use Laplace smoothing too, but for consistency with current unigram_counts logi
        # assuming non-zero probabilities for known words. If word is truly OOV, prob would be 0, leading to -inf.
        # For this exercise, let's assume all words in test sentences are in unigram_counts due to shared corpus for simpli
        # If not, a small epsilon or smoothing would be needed here.
        if prob == 0:
            # Assign a very small value to prevent log(0) for truly unseen words,
            # which would ideally be smoothed with V in unigram_counts/total_words.
            # This path means an OOV word appeared, its count was 0, and prob was 0.
            # Perplexity will be infinite if even one word has 0 probability.
            # For now, let's apply a minimal smoothing similar to add-one if it's not present.
            # This is a simplification; for a robust model, smoothing should be applied to unigram_counts directly.
            prob = 1 / (total_words + V) # Minimal smoothing to prevent log(0) in test set

        log_prob += math.log(prob)

    return math.exp(-log_prob / N)
def bigram_perplexity(sentence):
    words = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    N = len(words) - 1 # N is the number of bigrams
    log_prob = 0

    for i in range(len(words) - 1):
        prob = bigram_probability_laplace(words[i], words[i+1])
        log_prob += math.log(prob)

    return math.exp(-log_prob / N)
def trigram_perplexity(sentence):
    words = ['<s>', '<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    N = len(words) - 2 # N is the number of trigrams
    log_prob = 0

    for i in range(2, len(words)): # Start from the third word to form the first trigram
        prob = trigram_probability_laplace(words[i-2], words[i-1], words[i])
        log_prob += math.log(prob)

    return math.exp(-log_prob / N)

for s in test_sentences:
```

```
    print("Sentence:", s)
    print("Unigram Perplexity:", unigram_perplexity(s))
    print("Bigram Perplexity:", bigram_perplexity(s))
    print("Trigram Perplexity:", trigram_perplexity(s))
    print()
```

```
Bigram Perplexity: 121.04371938710919
Trigram Perplexity: 193.1720381277191

Sentence: Businesses leverage NLP for customer service automation, market intelligence, and content generation.
Unigram Perplexity: 429.19017208176314
Bigram Perplexity: 88.50201568010462
Trigram Perplexity: 147.37894539596263

Sentence: Researchers continue to explore new frontiers, such as explainable AI in NLP, multimodal NLP (combining text wit
Unigram Perplexity: 484.2384550961289
Bigram Perplexity: 119.49224954490444
Trigram Perplexity: 180.48323098956803

Sentence: Challenges in NLP include ambiguity (words with multiple meanings), sarcasm, irony, and understanding context-de
Unigram Perplexity: 543.1229913848848
Bigram Perplexity: 148.87822594146613
Trigram Perplexity: 173.81556644179994

Sentence: The vast diversity of human languages and dialects also presents a significant hurdle.
Unigram Perplexity: 496.54026241655544
Bigram Perplexity: 94.880228922332
Trigram Perplexity: 167.2465250452465

Sentence: Despite these challenges, the field of NLP continues to advance rapidly, promising even more sophisticated and h
Unigram Perplexity: 533.9545589742199
Bigram Perplexity: 133.80615624813902
Trigram Perplexity: 177.41527070173666

Sentence: This text is a placeholder to ensure the minimum word count is met.
Unigram Perplexity: 676.7259879478688
Bigram Perplexity: 147.08591143523475
Trigram Perplexity: 165.87966941305953

Sentence: It discusses the evolution and applications of Natural Language Processing (NLP).
Unigram Perplexity: 455.9153817270225
Bigram Perplexity: 140.77819534291638
Trigram Perplexity: 159.8312525539634

Sentence: It highlights the progression from rule-based systems to statistical methods, and finally to deep learning archi
Unigram Perplexity: 502.24257836991796
Bigram Perplexity: 112.63800821499673
Trigram Perplexity: 141.43441570266054

Sentence: Key applications mentioned include virtual assistants, chatbots, machine translation, and sentiment analysis.
Unigram Perplexity: 329.7445879173067
Bigram Perplexity: 72.78317311526037
Trigram Perplexity: 116.64441243324625

Sentence: The challenges of NLP, such as ambiguity and context understanding, are also touched upon.
Unigram Perplexity: 568.4747686011149
Bigram Perplexity: 139.89954567410322
Trigram Perplexity: 170.21187117400433

Sentence: The field is dynamic, with continuous advancements pushing the boundaries of what machines can do with human lan
Unigram Perplexity: 645.7819431424313
Bigram Perplexity: 140.5895105747803
Trigram Perplexity: 173.36204787602702
```

Comparison and Analysis

The trigram model generally produced the lowest perplexity values, indicating that it predicted test sentences more accurately by capturing richer contextual information. However, trigrams did not always perform best, especially when the training data was limited, due to data sparsity issues. In some cases, the bigram model achieved comparable or even lower perplexity than the trigram model because it required less contextual data. When unseen words or unseen N-gram combinations appeared in test sentences, the probability of those sentences decreased significantly. Without smoothing, unseen words would result in zero probability, making perplexity extremely high or undefined. Add-one (Laplace) smoothing helped resolve this issue by assigning small non-zero probabilities to unseen N-grams. Smoothing improved model robustness and allowed fair comparison across models. Overall, while higher-order models captured more context, bigram models provided a good balance between performance and reliability for the given dataset

Lab Report

Objective

To implement Unigram, Bigram, and Trigram language models and evaluate them using sentence probability and perplexity.

Dataset Description

A general English text corpus of over 1500 words related to language and NLP was used. The dataset was split into 80% training and 20% testing data.

Preprocessing Explanation

Text was converted to lowercase, cleaned by removing punctuation and numbers, tokenized into words, and sentence boundary tokens were added.

N-Gram Model Construction

Unigram, Bigram, and Trigram models were built using word frequency counts and conditional probability calculations.

Sentence Probability Results

Unigram produced higher probabilities due to lack of context. Bigram and Trigram produced lower but more meaningful probabilities.

Perplexity Comparison

Trigram generally achieved the lowest perplexity, followed by Bigram and Unigram.

Observations and Conclusion

Higher-order models capture better context but need more data. Smoothing handled unseen words and improved model performance.